

CONTENTS

Agent Colony — User Guide

1. What this is

2. Requirements

3. Getting it running

First time, on this machine

On a different machine

Making a desktop shortcut (no terminal needed day-to-day)

4. Day-to-day usage

Adding a project

Launching a project's agent session

Removing a project

Moving around the island

5. Reading the island

6. Where things are stored

7. Building a standalone installer

8. Customizing it

Common tweaks

9. Ideas for later

10. Troubleshooting

Agent Colony — User Guide

A desktop dashboard that turns your Claude Code projects into a little 3D world. Click a building, drop into that project's `cLaude` session.

Repository: <https://github.com/RevatureTyler/agent-colony>

1. What this is

Agent Colony is a small Electron desktop app. Each project folder you register becomes a building on a floating 3D island — trees, bushes, lakes, and a little person walking near each active building. Click a building to open a terminal running `cLaude` in that project's directory. Right-click to remove a project. Scroll to zoom, drag to orbit the island.

It is not a real game engine or a managed multi-agent system — it is a visual launcher. The “agents” are just terminal sessions running Claude Code; the app does not talk to Claude Code's internals, so it can only show whether a session is open, not what that agent is doing.

2. Requirements

Requirement	Why
Node.js 18 or newer	Runs the Electron app and its build tools
Claude Code installed and on your <code>PATH</code> (the <code>cLaude</code> command works from any terminal)	Each building launches <code>cLaude</code> in that project's folder
Windows, macOS, or Linux	The launch mechanism has a code path for each; Windows is the most tested
A GPU capable of WebGL	The 3D scene runs on Chromium's WebGL renderer (built into Electron) — any machine from the last ~10 years works fine

3. Getting it running

FIRST TIME, ON THIS MACHINE

The app already lives at:

```
C:\Users\Dudei\Desktop\agent-colony
```

```
cd "C:\Users\Dudei\Desktop\agent-colony"  
npm install  
npm start
```

`npm start` opens the Electron window. Leave it running like any other app, or close it — your project list is saved and reopens automatically next time.

ON A DIFFERENT MACHINE

```
git clone https://github.com/RevatureTyler/agent-colony.git  
cd agent-colony  
npm install  
npm start
```

Because it's a normal git repo, this is what “runs anywhere” means in practice: clone it, install dependencies once, run it. There's no server, account, or internet dependency beyond the initial `npm install`.

MAKING A DESKTOP SHORTCUT (NO TERMINAL NEEDED DAY-TO-DAY)

Windows: right-click `npm start` isn't shortcut-able directly, so either:

- Build the packaged app once (see §7) and pin the resulting `.exe` to your taskbar/desktop, **or**

- Create a `.bat` file on your Desktop containing:

```
cd /d "C:\Users\Dudei\Desktop\agent-colony"  
start "" npm start
```

and double-click that instead of opening a terminal.

4. Day-to-day usage

ADDING A PROJECT

Click the glowing dashed “+” landing pad on the island (it’s always present, off to the side of your existing buildings). A native folder picker opens — choose any project directory. A new building appears on the island for it.

LAUNCHING A PROJECT’S AGENT SESSION

Click any building. This spawns a new terminal window (`cmd.exe` on Windows, `Terminal.app` via AppleScript on macOS, `x-terminal-emulator` on Linux) that `cd`s into that project’s folder and runs `claude`. Use that terminal exactly like you would if you’d opened it yourself — it’s a real, ordinary Claude Code session.

While that terminal process is alive, the building’s window glows green and its little resident starts walking around. Close the terminal (or exit `claude`) and it goes back to idle.

REMOVING A PROJECT

Right-click its building. Confirm the prompt. This only removes it from Agent Colony’s list (`projects.json`) — it does **not** touch the project folder or its files.

MOVING AROUND THE ISLAND

- **Scroll wheel:** zoom in/out
- **Left-click drag** on empty space: orbit the camera around the island
- The camera can’t go below the horizon or too far away — this is intentional, so you can’t get lost.

5. Reading the island

What you see	What it means
A building with a glowing green window	That project has an open <code>claude</code> session right now
A dim, unlit building	Idle — no session currently running
The little person walking a loop	Session active (purely decorative — it does not reflect what the agent is doing)
Building shape/color	Randomly derived from the project's folder name — same project always gets the same look, but the shape has no functional meaning
Lakes, trees, bushes	Pure scenery, scattered randomly around your buildings each time the island is built
The dashed “+” pad	Always-present slot for adding a new project

Known limitation: the app has no way to see what an agent is actually doing (blocked, needs input, mid-task) — Claude Code doesn't currently expose that as an API. “Active” only means “a terminal we launched is still running.” See §8 for how to extend this yourself.

6. Where things are stored

Data	Location
Your registered project list	<code>%APPDATA%\agent-colony\projects.json</code> (Windows) — found via Electron's <code>app.getPath('userData')</code>
App source code	<code>C:\Users\Dudei\Desktop\agent-colony\src\</code>

Deleting `projects.json` resets the app to empty (no projects registered) without touching any of your actual project folders.

7. Building a standalone installer

If you want a real double-click-able app instead of running `npm start` from a terminal:

```
cd "C:\Users\Dudei\Desktop\agent-colony"
npm run dist
```

This uses `electron-builder` to produce a Windows installer (`.exe`) under `dist/`. Run that installer once; afterward “Agent Colony” behaves like any installed desktop app (Start Menu entry, its own window, no terminal needed).

8. Customizing it

Everything is plain JavaScript/HTML/CSS — no build step, no bundler, no compilation. Edit a file, run `npm start` again to see the change.

File	Controls
<code>src/main.js</code>	The Electron main process: how projects are saved/loaded, and exactly what command launches <code>claude</code> on each OS
<code>src/renderer.js</code>	The entire 3D scene — terrain shape, lake positions, building archetypes, color palettes, tree/bush counts, camera limits, lighting
<code>src/style.css</code>	Page chrome (header, add-project button, hint text)
<code>src/preload.js</code>	The narrow bridge between the browser window and the main process — add new IPC calls here if you extend <code>main.js</code>

COMMON TWEAKS

- **More/fewer trees or bushes:** in `renderer.js`, find `scatterRandom(22, buildTree, 0.5)` and `scatterRandom(16, buildBush, 0.3)` near the bottom — change the first number.
- **Lake positions/size:** the `LAKES` array near the top of `renderer.js`.
- **Building color palette:** `BODY_COLORS` / `TRIM_COLORS` arrays.
- **Camera zoom limits:** `controls.minDistance` / `controls.maxDistance`.

- **Add a real “needs attention” status:** this is the natural next step (see §9) — have each project write a small status file (e.g. `.agent-status`) that `main.js` watches with `fs.watch`, and surface that in the scene (a different building color, an exclamation mark, etc.) instead of the current “session open / idle” binary.

9. Ideas for later

- **Richer status:** a lightweight convention (a status file, or a Claude Code hook that pings the app) so buildings can show “blocked / needs input” instead of just “open/idle.”
- **Embedded terminal:** use `xterm.js` + `node-pty` to show the session inside the app itself instead of spawning a separate OS terminal window (this needs native module compilation, which is why the current version spawns a real terminal instead).
- **Multiple islands/”planets”:** group projects by tag (work, personal, client name) onto separate islands you can switch between.
- **Persist camera position** between launches.
- **Notifications:** a toast or taskbar badge when a session’s terminal exits, so you don’t have to keep the window in view.

10. Troubleshooting

Symptom	Fix
Clicking a building does nothing / errors	Make sure <code>claude</code> runs from a plain terminal on this machine — that's exactly the command the app runs
<code>npm start</code> fails immediately	Run <code>npm install</code> again; make sure Node.js 18+ is installed (<code>node -v</code>)
Blank/black window	Check for errors via <code>main.js</code> 's console-message forwarding — run <code>npm start</code> from a terminal (not a shortcut) and read the printed <code>[renderer] ...</code> lines
<code>npm install</code> warns about “allow-scripts” / electron not installing	Run <code>npm approve-builds</code> (or reinstall) so Electron's postinstall step can download its binary
Building an installer fails	<code>npm run dist</code> needs internet access the first time (electron-builder downloads platform tooling)

Generated for the initial build of Agent Colony — keep this file (`docs/USER_GUIDE.md`) up to date as you customize the app; it's a normal file in the repo, so it travels with the code.